

The user manual in plain English

A companion to `docs/USER-MANUAL.md`. That document governs; this one just explains. If they ever disagree, it wins. Everything described here traces to the manual or to `docs/BUILD-SUMMARY.md` — nothing has been added, and where the manual says it did not re-test something, this document says so too.

What this thing is, in one paragraph

The Alternate Histories platform generates made-up but carefully disciplined economic decades — ten years of monthly returns for stocks, bonds, credit, commodities, property and private funds, in a world you specify ("stagflation", "a deflationary bust") — and then lets you *live through* one as the investment office of an institution: watching the news arrive month by month, making a handful of big decisions along the way, and finding out at the end whether your decisions beat doing nothing. Everything it produces is reproducible to the last bit, and it keeps receipts.

Four honest warnings before anything else

The manual is unusually candid about its own limits. Carry these with you:

1. **The simulator is deliberately simple.** The engine that actually runs is called the *toy engine*, version `toy-v0.5`. It is not a secret research-grade model — the research-grade generator exists in the codebase but is not connected to anything you can play. What you drive is the toy, on purpose, so that every number can be checked.
2. **The generated worlds are not convincing models of history.** The

project's own sealed evaluation said so: the best generator beats its benchmark on the agreed test *and* understates crashes by roughly half and undercalls how long bad regimes persist. Nothing built on these worlds is decision-ready. They are a rehearsal room, not a crystal ball.

3. Private investments keep two sets of books, deliberately. Private equity, private credit and real estate are valued by appraisal in real life, so their *reported* returns are a smoothed, flattering version of their *true* ones. The platform simulates both, and the gap between them — biggest exactly when markets fall — is the point of the whole exercise, not a bug.

4. Your score is computed on the server, never in your browser. The browser draws pictures; a small server program does all the arithmetic that matters. This is a design rule, so that a score can never be faked from the display side.

If you have never used a terminal

Every command in this document is a line of text you paste into a *terminal* — on Windows, a program called PowerShell or Git Bash — and run by pressing Enter. The computer prints something back; the "what you'll see" line after each command tells you what a healthy answer looks like. Commands starting `uv run` use the project's own Python environment; commands starting `export` just save a value (an ID, a file path) under a short name so later commands can reuse it. You cannot break anything by reading output, and this guide has you work in a scratch database precisely so you cannot break anything by writing either.

Getting set up (once)

You need Python 3.12 and a tool called `uv` (it installs everything else), plus Node 20 or newer only if you want the browser game. Then, from the project folder:

```
uv sync --dev
```

What you'll see: a long install (it pulls in some heavy research libraries the first time), then a quiet prompt. — The manual notes it did not re-run this itself, because its environment was already set up; every command below was run against the environment this produces.

For the browser game:

```
cd app && npm install
```

What you'll see: npm downloading packages. — Also not re-run by the manual, for the same reason.

Check it worked:

```
uv run ah --version
```

What you'll see: `0.1.0`.

No account, no API key, no internet connection is needed for anything in this guide. The only commands that reach the internet are ones this guide does not run (live data downloads, the live text-to-world compiler).

Work in a scratch database. Everything the platform makes lives in one database file. Point yourself at a throwaway one while learning, so the real one cannot be disturbed:

```
export AH_DB=/tmp/ah-manual/ah.db
```

What you'll see: nothing — it just remembers the path. Every command below

includes `--db "$AH_DB"` so it uses this scratch file.

Make a world

A **world** is a complete written specification of a counterfactual decade — inflation, interest-rate policy, credit spreads, how stocks drift and swing, when crises hit, how the private-fund machinery behaves — plus a storyline that the number-crunching machinery is structurally forbidden from reading.

Four ready-made worlds ship with the project: `stagflation`, `goldilocks`, `deflation_bust`, `reflation_boom`.

```
uv run ah --db "$AH_DB" world build --preset stagflation
```

What you'll see: one line — a long ID ending `...000301`. That is your world's permanent name. Behind the scenes the document was checked against its formal contract and twelve coherence rules, any out-of-bounds numbers were pulled back into range, and a birth entry was written to the world's tamper-proof diary.

Ask the checker for its own account:

```
uv run ah --db "$AH_DB" world validate
```

What you'll see: `clamps=0 warnings=[] blocking=[]` — nothing needed correcting, nothing worried it, nothing stopped it. A clean world.

If you're curious what a world actually *contains*, print the whole document:

```
uv run ah --db "$AH_DB" world show 00000000-0000-4000-9000-000000000301
```

What you'll see: a structured page of numbers — the drift of commodities, the seed for the random-number generator, and so on.

One trap, worth knowing early. Almost every command here will, if you don't name a world or run explicitly, quietly use **the most recently created one**. Convenient with one world; a trap the moment you make a second, because every bare command silently switches to it. When it matters, paste the ID.

A word about typing a scenario instead. You can also build a world from a sentence ("inflation regime 0, rates path 0..."). Offline, this only works for fifty pre-recorded sentences — it is a lookup, not comprehension. The version that genuinely reads free text calls an AI model over the internet and needs an API key; the manual flags it as not re-tested. There is also a browser surface for this — see "The four browser windows" below.

Watch it simulate

Running the engine turns your world into an **ensemble**: many alternative ten-year histories (paths), each 120 months of returns for eight asset classes, all grown from one seed number so the whole thing can be recreated exactly.

```
uv run ah --db "$AH_DB" run 00000000-0000-4000-9000-000000000301 --paths 200 --seed 771204
```

What you'll see: one line — a fresh ID (yours **will** differ; it's a new one every time). This is the **run ID**: the name of this particular simulation. Save it under a short name:

```
export RUN=paste-your-run-id-here
```

What you'll see: nothing — it just remembers it.

What just happened: 200 histories were simulated, every output number was fingerprinted with a cryptographic hash, and an immutable receipt (a **RunRecord**) was written down — world, seed, path count, fingerprint.

Everything else in this guide is regenerated from that receipt.

The world's diary now shows both events:

```
uv run ah --db "$AH_DB" chronicle 00000000-0000-4000-9000-000000000301
```

What you'll see: two numbered lines — `birth` and `run`, the run carrying its fingerprint. This diary is append-only, enforced by the database itself: entries can be added, never edited or deleted.

****What you have *not* done:**** used the sophisticated research generator. The `run` command drives the toy engine and nothing else. The research generator also needs trained model files that are deliberately not shipped with the code, so on a fresh copy it cannot run at all.

Check the data going in

The worlds are calibrated against real historical market data, and that data has its own bookkeeping: every download is kept forever as an immutable **vintage** (a dated snapshot), so you can always ask "what did we believe on such-and-such a date?". A browser page lets you audit it without touching anything:

```
uv run uvicorn ah.dataconsole:app --port 8796
```

What you'll see: the terminal reports it is serving; open `http://127.0.0.1:8796` in a browser. — The front page answers "is anything missing or stale?": every data series with its coverage, gaps, freshness against its service-level agreement, and how much of its history is stitched-in proxy rather than the real series. Deeper pages show the raw series feeding each asset class (proxy stretches shaded), the private-fund returns before and after de-smoothing, and each series' full audit trail.

This console physically cannot write — a test enforces that it contains zero write operations.

Check the outputs

Four ways to look at what a run produced, from lightest to heaviest.

The figure page — one self-contained HTML file, openable in any browser, showing the fan of possible outcomes per asset, the do-nothing institution's value, the reported-versus-true toggle for the three smoothed sleeves, and the decade's episode annotations:

```
uv run ah --db "$AH_DB" inspect $RUN --out /tmp/ah-manual/page.html
```

What you'll see: the file's path printed; open it in a browser. — Nice property: the page is rebuilt from the receipt and re-checks the fingerprint every time it renders, so a pretty page is also a small proof of reproducibility.

The bundle — the single compressed file (under a megabyte, by enforced budget) that carries everything the browser game needs: the revealed path, the fan bands, the do-nothing institution's ledger, and the decade's news feed:

```
uv run ah --db "$AH_DB" bundle $RUN --out /tmp/ah-manual/world.bundle.gz
```

What you'll see: the path and a size around 31 KB. — One honest wrinkle the manual documents: the bundle contains *two* final values for the do-nothing institution, and they disagree (166.5 vs 76.7 in the manual's example). The `twin_ledger` figure is the real one — it comes from the full institution with a cash account; the `summary` figure comes from an older, simpler model that some display surfaces still run. Trust the ledger.

The QA console — a browser instrument for eyeballing whether a freshly generated world is *sound*, page by page:

```
uv run uvicorn ah.console:app --port 8799
```

What you'll see: serving; open `http://127.0.0.1:8799/worlds` . — The shelf lists every world with each of the twelve coherence rules shown individually; deeper pages check that no interest rate went below its floor and no price went negative, that the reported-vs-true gap actually exists, that the cash arithmetic closes to the penny across all forty quarters, and — with one click — that a run reproduces bit-for-bit. Read-only at the database-driver level: a write is refused by the machinery, not by good intentions.

The credibility report — not a live page but a generated HTML file, the admin's "is this world's arithmetic sane?" review. It regenerates the ensemble from stored seeds, computes decade statistics per asset, and flags anything outside a *declared* plausible band — bands one allocator wrote down in the code, so a disagreement is about a number rather than a vibe:

```
uv run ah --db "$AH_DB" credibility --preset stagflation --preset goldilocks --paths 200 --out /tmp/ah-
```

What you'll see: the path plus `(2 worlds, 2 flags)` ; open the file. — A flag is an invitation to look, not a failure; the report writes nothing and cannot block anything. The section worth your attention is the private-programme one: the commitment ladder, the call and distribution curves, the vintage stack.

The four browser windows, side by side

Window	Address	The question it answers	Can it change anything?
--------	---------	-------------------------	-------------------------

Data console	<code>http://127.0.0.1:8796</code>	Is the real-world data feeding the generator complete, fresh, and honestly labelled?	No — structurally read-only.
Build console	<code>http://127.0.0.1:8798</code>	Can I turn this scenario sentence into a valid world, and watch each stage succeed or fail?	Yes — the only one that can write, and only when you press Keep. Dry-runs store nothing.
QA console	<code>http://127.0.0.1:8799</code>	Is this generated world structurally sound — rules green, arithmetic closing, runs reproducing?	No — the database is opened read-only.
Credibility report	a file on disk, no port	Do this world's decade statistics fall inside bands a professional declared plausible?	No — it renders a file and exits.

The build console deserves its own line. Start it with:

```
uv run uvicorn ah.buildconsole:app --port 8798
```

What you'll see: serving; open `http://127.0.0.1:8798` . — Type a scenario,

press **Compile (dry-run)**, and watch a five-stage ledger fill in live:

prompt, model, extract, validate, stamp. Nothing is stored until you press

Keep, which saves the world exactly as the command line would, can record

an engine run, and links you straight to its row on the QA console. Every

attempt — including failures, with their raw payloads — is logged to a file.

Offline it replays the fifty recorded scenarios; ticking **live** needs an

Anthropic API key and a network connection.

Play a decade

Play runs on two programs: a small **session service** that owns everything

that scores (the authority), and the browser app that renders it (the

display). Start the service:

```
uv run uvicorn ah.serve:app --port 8787
```

What you'll see: serving on port 8787 — and it really is 8787, not the common default of 8000; checking the wrong port is the classic way to conclude the service is down when it isn't. (The manual notes it did not launch this exact command itself — an instance was already running — but it exercised the identical program in full against a scratch database.) Note this binds to the repo's *default* database; the manual shows a slightly longer launch form for pointing it at a scratch one.

Then the app:

```
cd app && npm run dev
```

What you'll see: a local address, `http://localhost:5173` ; open it. (Also flagged not re-launched by the manual, for the same already-running reason.)

The app loads a bundle, verifies its seal, shows the fan chart with the revealed path clipped to "now", plays the news wire in timeline, and presents each decision window.

How a decade plays. Time advances month by month, and only forward — the server refuses to rewind. At nine fixed windows (months 11, 23, 35, ... 107) you choose one of four actions:

- **hold** — do nothing;
- **derisk** — shift 10 points from growth assets to defensive ones, on the

liquid side only, because a private fund's value is not a dial you can turn;

- **leanin** — the reverse;
- **secondary** — sell a slice of your largest private-equity holding for

cash, at a 19% haircut. Distress has a price.

A decision, once made, is final — the server refuses to change one. Decisions land at the start of the following quarter, not instantly. You can choose to

see your private holdings on **reported** marks (the smoothed statements a real institution sees) or on **actual** (true) ones; reported is the institutional status quo, and its flattery is greatest exactly when things go wrong.

How you're scored. At the end, the server compares your institution's final value with **the twin's**: an identical institution, same world, same seed, that held at every window. The difference is your **alpha**. The server also decomposes it exactly — window by window, what each decision added or cost — and the parts provably sum to the whole, because it simulates every prefix of your decision sequence (a "hold" window contributes exactly zero, by definition: holding is what the twin does). It also reports whether you were ever a **forced seller** — out of cash and made to sell, the headline disgrace number.

Ranked sessions write one row to a leaderboard, keyed so that scores from different worlds, seeds, or scoring versions can never share a table, and your first play stands — a replay cannot overwrite it. One known wart: the leaderboard *display* in the dev app is broken (the development proxy passes session traffic but not leaderboard traffic, so the app receives a webpage where it expects data and shows an error). The server itself is fine; the fix is documented.

Prove nothing changed

This is the property the entire system is built around: any run can be reproduced, bit for bit, from its receipt.

```
uv run ah --db "$AH_DB" replay $RUN
```

What you'll see: two fingerprints — `stored` and `replay` — and the word `MATCH` . — That means the engine was re-run from the stored world, seed and path count, and the recomputed fingerprint over every output number is identical to the one written down at run time. Anything else is a loud failure, not a shrug.

The terse version, for scripts:

```
uv run ah --db "$AH_DB" verify $RUN
```

What you'll see: `True` .

Why this works: every random number in the system flows from a single seed through one well-specified generator, drawn in a fixed order, and nothing in the numeric path ever reads the clock. If the engine's numbers are ever changed, its version number must be bumped and the preset worlds moved to fresh IDs, so scores from two different engines can never share a leaderboard row. The figure page and the bundle also re-run this check every time they render — a tampered record produces a visible `digest_verified: false` , not a pretty picture.

When something looks wrong

"The session service is down." Almost certainly you checked port 8000.

It lives on 8787. Nothing in the project uses 8000.

You changed the server's code and nothing changed. The server keeps running the code it loaded at startup. Kill it and restart it. On Windows the usual Unix kill command doesn't exist; the manual gives a PowerShell recipe:

```
Get-NetTCPConnection -LocalPort 8787 -State Listen |  
  ForEach-Object { Stop-Process -Id $_.OwningProcess -Force }
```

What you'll see: nothing on success; then relaunch the service. — The manual verified the finding half of this; the killing half it deliberately did not run while documenting a live service.

"No worlds in the database yet." Commands that fall back to "the most recent world" fail politely on an empty database. Either you pointed at a fresh scratch file, or you skipped the build step. Build a world first. (Mistyping a preset name gets you the same shape of error, listing the four valid names.)

"Engine implements only 'toy-vo'." Someone set the world to ask for the research generator. The playable path cannot run it — by design today, and on a fresh copy of the code it cannot even be constructed, because its trained model files are deliberately not shipped. This is not a setting you can fix in the world document.

The leaderboard shows an error in the app. Expected today — see the wart above. Ask the server directly instead.

A garbled character in command output. One data command prints a dash that renders fine in PowerShell and as `?` in Git Bash. Harmless, but if you see mangled characters, send the output to a file or set the terminal to UTF-8.

Two final values disagree inside one bundle. Known, documented, and the ledger one is correct — see "Check the outputs".

You want to run "the battery" to check your install. Don't. There are two things called the battery. The everyday one (`ah battery`) currently has placeholder thresholds that nobody has ratified — it *cannot fail*, so a green result would tell you nothing and a number read off it is a number

nobody has agreed to. The research one is sealed machinery, not an install check. To check your install, run the core test subset instead:

```
uv run pytest tests/test_engine.py tests/test_validator.py tests/test_worldspec.py tests/test_stores.py
```



What you'll see: 80 passed in a few seconds. — The full suite is 2,226 tests and takes about twenty minutes; the manual ran it and everything passed. (One cosmetic formatting check fails on the current main branch, on two documentation files — a real, known, pre-existing condition, not something you caused.)

Words you'll meet

World — a complete written specification of a counterfactual decade: inflation, rates, spreads, equity behaviour, crises, private-fund structure, plus a narrative the numbers never see.

Path — one simulated history: 120 months of returns for every asset.

Ensemble — many paths grown from one world and one seed. The fan charts are percentiles across the ensemble.

Vintage — an immutable dated snapshot of downloaded real-world data. Nothing is ever overwritten; "what did we believe on date X?" always has an answer.

RunRecord — the immutable receipt of one simulation: world, seed, path count, and a cryptographic fingerprint of every output number.

Replay — re-running a simulation from its receipt and checking the fingerprint matches. `MATCH` means bit-identical.

Sleeve — an asset bucket the institution holds: five liquid (stocks,

bonds, high yield, commodities, listed property) and three private (private equity, private credit, real estate).

Commitment / call / distribution — you promise a private fund money (commitment); it draws the money when it invests (a capital call); it pays money back when it exits (a distribution). Promised-but-unpaid money is *unfunded*, and it can be called at the worst possible moment.

De-smoothing — statistically undoing the artificial calm in appraisal-based returns to recover what really happened. Its reverse — deliberately re-applying the smoothing — produces the statements the player sees.

The twin — the identical institution, same world, same seed, that does nothing at every decision window. The road not taken, kept alive so it can be measured.

Alpha — your final value minus the twin's. The one number the decade boils down to: did deciding beat holding?